Active InferAnt Stream 011.1 ~ April Agents Bring May Simulations: OpenM

ActiveInferAntStream | April Agents Bring May Simulations: OpenManus, RxInfer.jl, and ... | 1:28:01

All right, welcome everyone.

It is April 2nd, 2025, and this is Active Inference Stream number 11.1.

And I'll begin with a GitHub push.

Looking forward to everyone's updates and ideas and questions in the chat.

This should be an interesting stream.

So, the title is April Agents Bring May Simulations.

OpenManusRxInfer.jl

and Agent Laboratory.

I have a few things prepared, a few software implementations we can play around with.

Also, some topics to explore.

Not sure how any of the demos will go live.

However, it's the hope that a few things will be pretty interesting

and we can fix and explore along the way.

So, April Agents Bring May Simulations.

Or as Alexandra just pointed out, April Agents May Bring Simulations.

There are many exciting developments and advancements in the multi-agent synthetic intelligence systems space right now.

In a pretty broad sense.

Short-term, medium-term, all the terms.

Let's explore a few articulating, complementary, interesting kinds of multi-agent frameworks.

Autonomous frameworks.

Look at different ways of looking at what makes agentic.

What kinds of systems engineering are being done within and among the agents.

All these kinds of orchestration questions.

Looking where we're at instantaneously in April 2025.

And more broadly.

So, there will be some jumping around since some of the pieces take different amounts of time to load or play out.

So, again, I'm not sure what exactly will come through.

That'll be part of the fun though.

On all of these branches, I've been having some interesting times.

There are three cursor windows open.

So, starting with the two LLM type models.

Right now is running in this terminal.

The work of Samuel Schmidgall, Agent Laboratory.

I'll read the overview.

Agent Laboratory is an end-to-end autonomous research workflow meant to assist you as the human researcher towards implementing your research ideas.

Agent Laboratory consists of specialized agents driven by large language models to support you through the entire research workflow.

From conducting literature reviews and formulating plans to executing experiments and writing comprehensive reports.

This system is not designed to replace your creativity, but to complement it.

Enabling you to focus on ideation and critical thinking while automating repetitive and time-intensive tasks like coding and documentation.

By accommodating varying levels of computational resources and human involvement,

Agent Laboratory aims to accelerate scientific discovery and optimize your research productivity.

This has an associated paper describing more.

So, I got it to work initially.

Output a really awesome 17-page paper, images, equations.

I have the PDF around.

Then I started modifying it, trying to get more logging, get more access to it.

And kind of went too far.

And now I've just pulled back to the main branch that they've provided.

Just to show this experiment while this one's running.

And then we'll see how long it takes.

Hopefully it'll finish within the stream.

This YAML file, which they provide some initial examples of.

I made a variant that has the following sections.

Copying it over with control-I, write a new YAML compliant file that has this semantics.

The goal is to develop a partially observable Markov decision process, POMDP,

implementing active inference that is in a thermal homeostatic setting.

So, kind of the classic air conditioner, do-nothing heater temperature.

The API key for OpenAI is put here.

And the model is provided for the LLM backend and the review, I guess, separately.

Language.

Here's different stages.

So, I found that when larger numbers were used, there were some interruptions.

I don't know if that was because of stepping away from the computer.

So, these are relatively low levels of steps, but it'd be conceivable and interesting to see what happens when literature review is huge numbers or what happens there.

How to use a structured knowledge base for literature review.

It's also possible to run multiple labs in parallel.

Then there's these solver steps.

I wonder if that will throw an error later.
We'll see.
These solver steps, which are described in the MLE solver and the paper solver.
Load an existing save.
And to compile all the way on through the LaTeX PDF output.
And then this copies over the notes format.
And it's doing a pretty multi-staged effort.
The package setup did have a few other packages that on my computer I needed to set up.
But it was pretty easy to get through those with help of cursor 2.
And get the virtual environment.
And at least get it along the way using the API key.
So, this is pretty cool.
And, again, keeping with this sort of just snapshot.
I'm sure with slightly different setups or just different settings, these are all really functional.
I'm just showing how they're running for me at this moment.
And hopefully come across useful outputs.
But big picture.
Agent laboratory.
Really cool.
And partnering it in this sort of multi-agent, autonomous agent, LLM type coordination.
In the second window, we have OpenManus.
So, I'll read the description.
Manus is incredible.
But OpenManus can achieve any idea without an invite code.
It's a simple implementation.
So, we welcome any suggestions, contributions, and feedback.
Enjoy your own agent with OpenManus.
So, look at the demo.
Okay.
So, we'll see.
Here's videoing into their setup.
Describing the task.
Basically carrying out this task within the repo that it's running from.
So, here.
There was.
I started with a request.
Write a research report on Davis, California.
And I output this after 20 steps.

But it popped up a browser.
And now running an enhanced version.

To see if it's possible to get a longer report.
So, it could just be how I'm running it or setting it up.
It could.
But just sort of conceptually, we'll look at the architecture of how it's set up.
Even if nothing too impressive comes from it.
And those will be the two LLM type processes running.
Here we are back in the PMDP research.
Review writing.
Code writing.
We'll see if we can tell what the output.
But there are subtasks.
To what extent these are explicitly described.
Or to what extent there's an initial plan or hierarchical plan formulation.
How many steps that's taken in.
How general the prompting.
All those kinds of things.
A lot of degrees of freedom.
Just seeing how it works here.
It's just interesting.
And then also, again, openManus.
Where here, let's start this one again.
With a new.
But did it even result.
Let's try to improve the logging.
On openManus.
Then we'll go to the third window.
Which is the rxinfer.jl.
Given this type of quote.
Successful output.
We ensure we are capturing.
All outputs of.
Who will use better.
And enabling.
Running tools more intelligently.
In order.

To achieve.
Sequential.
Incrementally.
Improving.
Outputs.
Outputs.
General case.
All right.
And then the third window.
Is going to be.
The.
Rxinfer.jl.
Rxinfer.jl.
Examples.
Repo.
So here's the openManus.
Now in the third window.
Will be.
A.
Para repo.
For Rxinfer.jl.
rx infer which is uh under some very interesting and good open source development as part of this growing interesting rx infer ecosystem including also recently rx environments not going to be probably covered here and also this recent rx and first server so opening up the possibility for different languages to use api calls into these rx and for type pipelines through their their server and uh this might connect with some general topics and multi-agent just depends what people write in the chat and where these tools sort of take us okay
using cursor 0.48
using for the most part claude 3.7
agent mode
all right started with the github push and let's just check on each of the uh
uh ongoing language model uh windows
then look to some of these recent examples from rx infer including how they integrate with neural networks and see if we can generalize that or connect that also to these llm type methods
um and yeah see where the different work products from these more llm tool use agents where that gets us and where we get with okay here we go
yep
mle max steps

let's just do one max of everything so that it happens from all the stages
okay i'm gonna um make a new api key
to put into this

moving it off screen so i can put in this api key this way and save the file
So that'd be one way I thought of modifying it, having a environment file or other method for the API. A lot of flexibility with which LLM and how to call it. So that could just be one thing and then here's okay. So let's run that again. AI lab repo is the script and then it looks to the location of the YAML file. So AI lab repo. All right, looking at it while it sets up, gives a different language default model, maybe overridden. The laboratory workflow is a 400 line, 500 line method. The archive agent. Here's the argument that parses, argument parsing for the YAML.
Parsing a YAML and then main. Or the main logic of this main script that's now playing out here.
Sets the LLM backbone. Determines whether it's in human mode or not. We can try that, look at the YAML and flip that value. Compile PDF. Go from the LaTeX plaintext all the way through at least trying to compile the PDF. Load in the previous workspace. If there are parallel labs, what to do in exception, failure. Whether there's an archive research agent and the indices of the lab. Then there's those limits states on each of those steps, number of times to iterate through these different steps, um, and how to parallelize. Getting the API keys from the YAML, whether the OpenAI key or the DeepSeq key. Has a logic for if it's in the human interactive type mode, asking questions or whether it uses the YAML information. Language, human in loop mode, possibility for different stages to have different human in or on the loop. Agent models, which LLM to use for different phases. Here it looks like this is a set sequence of phases. It could be specified in a ton of different ways. Parallelism instructions. Some exception clauses. Checklist. Practical. Make a better config system. YAML. Advancements. Make the ability to have agent build on top of the research run agent labs parallel. I'm not sure if that's to do or what was done. So this is going to keep running. Hopefully we will not hit that same error. Should be a little faster, but still might be a couple minutes on that one. All right, now to this one.
So something was happening in the OpenManus.
So we asked cursor in the agent mode to enhance our ability to understand because the research report was not very long. So maybe it's just handled differently with the database and the shared state.
And this is an example of kind of a classic and durable type of trade-off in multi-agent modeling.
So we have different kinds of information interactions ranging from this just kind of informally implicit, multi-agent hub and spoke type computational architecture, if not Bayes graph, and spoke type of data. So we have different types of data. And this can be in certain digital settings, and even probably some physical ones too, describe the possible layouts of how to organize shared state. In contrast with where there can be direct interactions, like interactions among nest mates, as well as with the shared environment.
And in the context of doing these computational agents, it is like, what is being remembered and modified? How?

How? Where's that statistical updating of a probability distribution, like the Rx and Fur side of things?
Where is it some other kind of model update? Where is it context for a language model? Or
where is it context for a language model? Or to what extent is that update private within that agent?
Carried like sort of like a nest mate level memory? To what extent is it accessible by different processes?
And it kind of just ends up being just in some particular way that's very situational. So this is just showing a few of the tools.
And hopefully with people's comments or suggestions, let's just see how many minutes it takes to look into a few different functions,
or at least connect how certain package analyzes this, connect that to ways we can integrate amongst these different frameworks.
Okay, now we have...
Agent laboratory is running in the first one. Open madness. We ask for these improvements.
Agent laboratory is running in the first one.
Now run the improved open madness.
So sometimes there's a really obvious way or an entry point.
Um, and I think good code-based design can go a long way. I know there's a lot of cool work being done with the cursor rules and the MDC files,
uh, as well as the MCP server calls. Those are all things to, uh, look into, but even, uh, uh, a code base of this level.
Sometimes it's a little bit hard to even know, uh, which, which script to run or which entry point.
So sometimes it's even useful just to ask in the agents on the right side to even write that
as it, as it needs to be.
So let's see, um, let's see, um, let's see.
And also to update that experience. Like, okay, I don't want to do any terminal, uh, extra arguments.
Just make it so that this works.
Um, so let's let cursor work on that while we look onto the third
of the three, um, modeling approaches, the RxInfer.
Okay.
So the RxInfer examples repo is maintained by the Reactive Bayes organization,
um, lazy dynamics fellows. And what I have in the doxology fork is a, another folder called support.
And right now it has a few, two scripts. There's setup.jl, which is at least a starting point for
setting up the packages and, and gives a, a scaffold for adding other packages that you might
want to install or something like that. And then importantly, the notebooks to scripts
script. So this can be run from here with Julia notebooks to scripts, the file name, Julia space,
the file name, and takes the documents that are in the original repo.
So this can be run from here with Julia notebooks. And then, uh, the, uh, the, uh, the, uh, the, uh,
this script converts those into a matching scripts folder. So we will look at, uh, two or maybe more of these
script, uh, uh, analogues of some RxInfer examples. Um, they're a little bit more flexible to work with.
It's possible to break them out into multiple components. And some of them probably run a little

smoother in the notebook setting. Uh, but by and large, they, uh, all work to, with just a, a few tweaks or ordering changes. So we'll look at some that are already, uh, some variants have been made of, and then just drone or, um, some other interesting one. Let's, let's explore it. Okay.

Agent laboratory continuing.

Um, cursor improvement of open Manus proceeding on this site. So it's like their agent orchestration, their microservice, um, process orchestration, all, all these really advanced, um, services and software, perplexity, et cetera. Pretty interesting how they, how they do it and, and, and surface it. So we'll leave that going and wait for it to complete while looking at the RxInfer, uh, examples.

So, um, first example will be, uh, the recently written recurrent switching linear dynamical system. This one, uh, where we're getting to with it, just to sort of put the, um, and this is not, not to say that this is perfectly statistically optimized, just to show some of the expression capacities of where, where we can model. And then from there figuring out, um, is RxInfer, where, where, where might it be at different stages and different projects, and what if probabilistic modeling in, uh, in, uh, in, uh, in, uh, in, uh, in, uh, in, uh, in, uh, with different machine learning pipelines, like where, where and how might, and what if probabilistic modeling in, in fusion with some more dynamical systems type, uh, time series modeling, neural networks, networks, LLMs, uh, all these different architectures of statistical models.

Uh, it's, um, where, how do we compose useful systems for variously more on the recognition direction, doing, uh, latent state inference from noisy data, which is a commonality of, of the models that, that will be looked at. So in this, um, you know, or using it to recognize latent states from noisy data and or generating predictions from latent state estimates. So here in this setting, there's a noisy sequence of measurements and at a, uh, discrete layer, there's, there's a switch between two regimes, one and two, where the second discrete state has this higher amplitude oscillation.

And so there's a switching between these two modalities and we can explore how, how hard is it to have a three state switch model? Um, so at the sort of faster layer, there's this time series interpolation, uh, auto-regressive type model, which we can look at with the app model statements. And then at the slower level, there's this linear. Um, and then the other example that has, uh, a lot to do with this first example is, um, uh, three-dimensional in time Lorenz attractor with noisy sampling coming from it.

And in this, uh, and in this, uh, example, integrating neural networks with flux.jl, there's a two-step process that has a lot to do with the continuous time, time series interpolation, and the slower switching model. Here at the first level, there's the learning of a, uh, uh, statistical model of the Lorenz attractor state space using a flux neural network in Julia. Then probabilistic parameters, according to a at model statement in RxInfer are used to model certain distributions of that trained neural network. So it's like a common theme is, so here's now the three dimensions of the Lorenz attractor. So the blue line is the inferred from the actual Bayesian posterior. Um, in terms of not possibly miscalibrated. It's not to say that this makes this an accurate posterior assessment. I think there's a lot to, um, to, to look at in there, but these are posterior estimates that in certain conditions or situations or data sets can give better estimates and other alternatives

that have, for example, in this situation on this dimension for the Lorenz is a, uh, parametric estimate in terms of a mean and a variance of this posterior distribution constrained to the Gaussian family of just mean and variance parametric statistics. Uh, rather, and that can be computationally implemented, typed, simulated, et cetera, in a number of ways with an explicitly distributional analytical nature, knowing that it's going to be this normal distribution. Now, if the underlying state is, is bimodal, then there are all the sorts of attractor states depending on where it goes and how the optimization unfolds, like whether it over disperses to capture the mean or whether it sharpens to go onto the median, like a local optima. So, but all that being said, it gives this parametric estimate in terms of an explicit mean and variance, which can be used in any statistical method for what would otherwise be distributed throughout the weights of like the neural network with many parameters needing to be simulated for inference. Whereas with these more distilled summary statistics on the posteriors, it can be compressed to a very small, um, scale. Even though as, as, uh, more parameters are added, the fit might be better because the neural network might be able to, to know that there was some more structure than just the, the two parameters that are allowed to be specifying this distribution.

So that's kind of the, um, topic that I think in, in a lot of ways, regardless of what the agent orchestration layer is, which this is giving us some windows into, there's sort of this orthogonal analytical or behavioral scientific framing of this information environment using these, uh, using probability distributions to model outputs of other models. Some places and motifs where that could be like parameter on parameter on parameter, or it could be LLM on LLM on LLM.

So that's the, um, all these different chains of, uh, of interactions between models that are more like, let's have a ton of parameters and try to interpolate a function and pick the size of the model out of convenience or, or compute. Like why 7 billion parameter language model? Because maybe, uh, just in the short term, it's easier to fork or use the, the training methods that are known or whatever it may be, but use a certain given large number of parameters, choose your own or some other common number, and then hope that there's enough degrees of freedom and in the curriculum of the training that it's, uh, that it interpolates and learns the space and does the good function approximation. And, and you don't have to do an analytical description of like the information space, yet it can learn a lot of the important structural features. Um, whereas for example, uh, explicit probability distribution over words, which is kind of has a functional encoding in the, the larger, um, many parameter type, LLM type, learn all the statistical associations, including higher order ones among different textual inputs, and then learn the, the specifics of that from, from the training data. But specif-, letting those weights and coefficients amongst all of the billions of parameters become fit through the training. Uh, in this app model type, RxInfer, um, setting, and then also in, in the textbook group, there was, um, the, um, one person was interested in posture, so we modified it. So maybe we can modify it another. Of gray is the proprioception, noisy signaling coming from the body. Then there's the perceived and the

actual

ail ail

tractor systems could be sampled from, or mixtures of systems could be

multiple ways of modeling the generative process that's giving rise to these

noisy observations. It doesn't have to be just fit with Lorenz, but similar theme.

Look at the app model that defines the probabilistic part of it.

Here's the state space model which can be in certain places and ways

run in situations where the larger neural network might not necessarily be able to operate.

Like all the variables that have to be stored within a semantics of this being the arguments passed to

this SSM. These can be interpretable, true variance, posterior estimates,

explicit composition of different statistical distributions. There's more to it for

logging and for coordinating the calls and the orchestration of how these app models get used

in the rhythms of other API calls and program logics. But the app model is kind of this statistical

kernel that is the one that gets parsed and rendered and used as the underlying declarative base graph

for the RxInfer model that then gets a certain scheduling of either the standard sort of pass over

all of the nodes and update them in this order, imperative style, update this and then do this and

then do that. And then to what extent that scheduling of updating strategies on that base graph,

that is what reactive message passing in principle kind of opens the degrees of freedom around. So it's like,

well, this can be under this timing, this one can be triggered by this or react to this rather than the whole graph

being either updated sort of in a sweep or just in any given fixed for loop type computational execution

or approach to that distribution. So this is the joint generative model of the state space model inferred from the neural network.

So the of the, uh, from the noisy Lorenz data.

So, um, let's check the other. Okay, Agent Laboratory still writing.

Cursor finished.

Well, commands to run to run and log the demos written.

Hello, Zig.

I'm going to go to the neural network.

Train the neural network on the Lorenz noisy data.

So using just a ton of parameters, throwing it out the problem.

Training the neural network.

Training the neural network.

ail

This is what we get. There should be at full function logging of the vast
sequentially improved actual plans, documents, research outputs, etc. logged and enacted by this system.
Agent laboratory is still going.
Okay, that was the neural network. Learning the neural network and then describing parameters of a specific app model.
So those were two of the... but let's look at a new one like drone dynamics.
Alright. So I've done this one. Let's see what it results in. Let's see what it looks like.
So this is in advanced examples.
Drone dynamics.
Notebook.
Drone.gif.

Drone.gif.

2D drone.

Drone 3D multi.

Looks awesome.

So if we can recapitulate this and start extending it a little bit and understand the app model.

Add in some logging.

Look at some other visualizations.

Adapt it to foraging and the honeybee or something like that.

Like there's so many.

So this is just as we watch these three processes play out.

Which again under total.

No.

Make it happen.

And do it.

Comprehensively now.

These.

You know.

Two weeks later.

The level of function could be.

I guess somehow.

Deteriorated in some way.

But.

Considering just for the most part.

Largely improved.

A lot of the things that I'm doing.

Certainly someone with different or more development experience.

And or.

Another more advanced version of cursor.

That could just iterate a few more times.

Or a little bit better with coding.

So it's.

It's pretty wild.

And.

I'm excited to see like what people.

Do with.

It's not just vibe coding.
It's.
It's more and different than that.
I think in.
In ways.
Or at least it can be.
Okay.
Quick scan over the drone.
Example.
These examples demonstrate the use of.
RxInfer for motion planning.
The animation show the inferred trajectories.
From probabilistic inference.
Rather than simulated executions.
For more realistic simulations.
Especially in the 3D drone example.
The model would need to be.
Extended with reactive environment.
Hashtag Rx environments.
That responds to the drones actions.
During plan execution.
If you're interested in collaborating.
On a more realistic implementation.
Please open a discussion.
And let's work on it together.
And that links to.
A discussion.
On reactive base.
Okay.
Let's quickly.
Look.
Through it.
Import packages.
Environmental features.
Like gravity.
Physics.
Physics.

Inertia.
Radius.

Properties.
Of the drone.
Aerial properties.
The drone.
State of the drone.
Vector.
Of these variables.
Model specification.
Location.

And that's the.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
The drone.
Model.

Location state transition based upon this declarative statistical style.
Graphical model specification for the drone model that's going to get rendered
up and done all these ways with the RxInfer using the at model macro.
Planning loop included inside of the at model macro for the time horizon involving
something like rolling out the prior on consequences of action, physics related,
with the state transitions.
But how does it select action?
Then, probabilistic inference.
Not sure what unscented means here, but some kind of state transition approximator.

This is an interesting function.
I think it'll be like, is it really this easy to make a wrapper function called move to target?
If so, that feels like a really high level function, even for a 3D nav system to be able just to be like,
move the target and have it have this definite-ness.
That'll be interesting to look at how that happens.
So, hopefully getting the script running will get us to understand all these different visualizations,
and maybe make a few new visualizations just using the script and generalizing it from there.
Let's check on the LLMs, then continue with the drone.
Alright, Agent Laboratory continues to use this.
We had some really fun, interesting discussions in the Discord.
So, like, just taking a little snippet here.
These are LLM costs.
Postdoc.
Let's take a fresh perspective in developing our POMDP model for thermal homeostasis,
focusing on agile design that prioritizes user-centric features while maintaining strong mathematical integrity.
Here's a newly formulated plan that emphasizes practical user engagement and theoretical robustness.
Three action states of the POMDP.
Five continuous band on a continuum making a discrete state space five option latent space.
So, it's laying out different aspects.
Certain parts of it are repeating things.
It could just be logging.
It could be other ways to do the prompt engineering.
Like, this just looks like it's repeating things right now.
Maybe it's actually asking if we're ready or something.
But, yeah, I think that'll interrupt.
Let's see.
Okay.
Something is happening.
Let's leave that highlighted and return to it.
Here looks like the cursor agents.
Make the updates and run it.
And fun it.
Okay.
Back to the drone.
So, we looked at the readme for the drone.
This is in the scripts folder, which basically takes the notebook that we looked at with the interspersed comments and pulls out just the Julia code.
So, right click on the code.

Open an integrated terminal.

Open an integrated ail

broader service of living and computation. Start out with just running it as it first converts.

Julia Drone Dynamics. It may need a slight reordering or it could just be like my environment,

but we'll see. One feature that comes up a lot in the RxInfer group and in learning about it is just like

this very Julia-esque usage of statistical variables very close to as they are analytically,

isomorphically, I mean ideally, and learning the syntax of what is allowed in plausibility

where is it speculative realism, distributional specification, where and how does the

does the current ability to make different kinds of nodes and implement different kinds of edges in

RxInfer, which could be more general or robust in the future. I'd expect so just from an open source

software development side, like what kinds of app models will what kinds of statistical behavior

arise from, what kinds of logics and functions like go to target are being

put within the app model, what's in the logic of updating and calling

different nodes. There's just a lot of interesting areas and spaces for taking these functional examples

and if if if and being clear about where where are we having problems in implementing these examples, like I don't expect it to run the first one

because it was a notebook but then it hopefully is just one or a few

prompts away and then we can start taking these things apart, looking at it and making more logging,

testing different variants. Please update this to run.

to run....

it just hangs, it was converted from notebook.

Break it up, out, into any separable modules properly invoked if you think it best.

Okay, looks possibly...

Let's just see what happens if we say move ahead.

Okay.

Let's...

This one seems to be hanging again.

So here, one affordance that they sort of point to is new chat.

So this is back to OpenManus.

Here is what we get.

We can now use a chain of OpenManus calls and tool use on incrementally writing reports and doing dispatch of document writing.

Then stitching together as editor at the end.

Okay, this is still happening.

So that took 1800 seconds.

Okay.

It's loading some information.

This is like...

This is getting into the...

Yeah.

Stuff is coming out.

This is one of the big open questions that hopefully we won't get too off the rails with in this...

With the tools.

But even with the...

And perhaps even especially with the haphazard and just open source grabbing these tools.

But even if it were a much more well orchestrated, documented approach, even that would be an especially important like what is really being empowered in the environment that I'm running this in.

So, where are different kinds of real consequences possible?

It's just...

It's pretty interesting.

And how does that relate to the actual technical basis of what the model is and does and how it's interpreted?

Okay.

Okay.

So, let's move some...

Drone core.

We can use perplexity to make a drone core image.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Okay.

Yeah, so in the time when it wrote the 17 page PDF....

Okay....

Yeah....

Okay....

Yeah....

studio Ghibli.

Okay.

Yeah, so in the time when it wrote the 17 page PDF, so it didn't output the other discourses, but a lot of more stuff was happening. But it just saved in the time that I previously ran this just a 17 page PDF on this thermal POMDP model and concept. But it really did have files like this file, partially haphazardly outside of its lane, but not few files were made in the same way. Hugging face datasets really may be coming here. There really were Python scripts that did have some outputs and some source code was written. So we'll see where that goes. And I think that's another sort of general topic is this file structure handling. To what extent should more

relational databases and general high performance databases, and maybe they're already being used in in different ways. To make some of the handling of the reference of these different system components more facilitated. Doing it locally, like all of these examples, has a lot of interpretability. It's like very hands-on, but for high reliability there's some other methods, but those have their own sort of trade-offs. So okay, so we got... OpenManus being improved by cursor to make more sequential incremental state sharing amongst runs.

So let's just run the main...

This is...

Let's run...

This is the initialized script.

Activate the virtual environment... yes.

Run OpenManus... yes.

Okay.

Okay.

Okay.

Drone example.

March is on.

Pop-out terminal.

So it is happening.

Could be hanging in a really useless place.

Okay.

Code execution.

In the agent laboratory test train split.

Loading some data from HuggingFace.

Lab number two is the index of that lab.

So it's like...

Where...

I don't know where...

This is happening in the doc cache.

So where do different files get stored?

Where does that get stored in databases, etc.?

Okay.

Okay.

Okay.

Okay.

Looks like some kind of error.

But what was very interesting to see is...

This...

So, hmm.

This is a...

I just wasn't sure what level of code this even was in terms of errors happening in certain tasks.

Blurring with written scripts because the scripts for the tasks are in this repo which also has edit access to these tools and to cursor.

So it's just...

It's probably not the most crystallized high reliability setting but it has this sort of open-ended evolution element with the agent laboratory especially.

Um...

Okay.

Open Manus.

Common Cursor Pathology.

Let's...

Let's...

Let's do something totally different.

Use the open...

Let's just start a new chat.

Fix this.

So that...

Main.py works...

Fully functionally as it should.

Okay.

Okay.

RxInfer Drone Model is running.

Postdocs....

Just....

Postdocs just being postdocs.

2D.

Okay.

While that's running.

Add more logging to the outputting so we better with emoji structuring.

Understand and can report on the timing and resource of different stages.

Okay.

So, RxN4 at least is doing some logging here.

Running the...

DroneDynamics...
Simulation.
Break out the...
Core Program...
SPAC and Logic...
Flow...
From the...
Statistical...
Description...
Of...
The G...
Model...
Statements.
Okay.
Errors from Julia.
Okay.

Okay.
Errors from Julia.
Okay.
Errors from Julia.
So now with the errors from the logging from it.
Fix this and add more there.

Okay.
Now main is working.
Alright.
So here's what the OpenManus functionality.
Just running the main.

Enter your prompt.
Enter your prompt.
I am in a...
LiveShift.
Now I'm going to click on the logging from it.
Now I'm going to click on the logging from it.
Fix this and add more there.
OpenManus.

Okay.
Now main is working.
Alright.
So here's what the OpenManus functionality.
Just running the main.
Enter your prompt.
Enter your prompt. I am in a live stream on multi-agent dynamics.
Research and report on with five paragraph essays and code stories
about what kinds of state-of-the-art methods it would make sense to
comprehensively understand in April 2025.
Oh, already.
Let's see what time it is.
Okay, let the open manis repair.
Still writing
PhD dialogue. Now it's into the storying of the
paper.
All right, back to the drone.
Working on drone dynamics.
So hopefully that even though I'll do about 10 or 15 more minutes,
these are all right in the zone of just hit and miss right now.
I'm just sort of, that's why I'm showing from a more reset state.
But I think people can report back and obviously we will see what is really possible
that we're going to do with making these systems a lot more higher liability in multiple
flexible keyways
so okay so yeah this is coming from open manis this is with the browser
so let's pop this out yeah incredible
okay it popped up this window with these analysis boxes
brought here
is going to be the terminal
this is manis selecting these tools is it the uh you know is it the best possible way to look at
this action selection or you know how to interact best with sites without so many degrees of freedom
but that's a pretty cool outcome so let's let that browse and and and consider the watershed moment
while agent laboratory is writing the thermal p omdp and now we have the improved logging
rx infer drone form
let's do another drone
script same total functionality with fewer time steps
well let's just let let let's reduce the number of time steps
in

our simulation
okay
add to chat
add to chat
and fix this
so here's the step 11
so that's what that was the chain of of tool use that was leading to that website the news website
okay
all right it has popped up
family center dot meta dot com looks like possibly slash brazil in portuguese
unexpected but okay
okay grab the tension again with that paper continuing on in the open ai
so
variably successful
um modification using llms of
rx infer code
so that's one one reason it's fun to do this script first off it allows a lot more articulation and
separation between different parts of the code and better logging for for when things are not working
it can break things too
but that's the helper script so as long as they keep the main rx infer
examples notebook functional or or similar reference models of code
then it's a lot easier to regenerate functional scripts and start modifying from there
yeah
yeah i'm going to do about 10 minutes more
so if anyone has any comments or questions they want to have mentioned in the last 10 minutes write them
okay let's look at how tool use really
output
make sure

running just 2d quick. So let's see if this will successfully save an output.
It was working a little more earlier on with the numbers in the outputs, so let's see if we can at least recover that. If not, save into... but once we have the scripts and the ability to to modularly break it out in pretty simple steps, then there can be different logging and saving.
It can be okay, we'll now just deposit that vector into this matrix form and then just keep on working from there. So there's a lot of ways to use even just the state space clarity that making these probabilistic models can provide. That's kind of the systems modeling part that's always really interesting. So 15 steps deep, possibly just... what did we even ask for? What time is it?
Overkill, right? But general.
So Open Man is probably not even pretending to be otherwise than such. Seems like a little bit less of a functional version. It's... but it's a great direction to at least point in that direction.
Agent Laboratory has the associated paper.
Let's hope that it will finish in the last...
Again, things are happening with Open Man is so it has that sort of tool architecture.
Paper is still writing over there.
16 out of 20 Open Man is.
Drone Air.
Yeah, and also it's helpful to have the notebook form available in the codebase.
So then it can be referenced say, okay, this was working, especially early on.
Yeah, I feel that this will not finish and render PDF, but the PDF
that it did render previously was... keeps on going to this website.
Let's see what one it opens next. The PDF was stymied probably one or two times,
but then generally was rendering totally fine.
So they've used a bunch of different methods.
It's really cool work.
Yeah, multi-agent, active block prints, where we've for a few years sort of with ebbs and flows, and talked a lot about these topics, about approaches for cognitive modeling,
ways to take procedural approaches, ontologically driven approaches to modeling, connecting that with data integrity stuff like from block science with different and other requirements engineering,

p3if, these kinds of modeling layers.
Oh, it detected that the browser window is closed. Maybe that shouldn't be done next time.
Let's see if it can fix the drone example in the next seven minutes.
Let's see if anyone has any last comments. Go for it.
Okay, I'll close that. Oh yeah, the drone core.
Images and perplexity. Not sure why they kind of broke it out. It was nice when it was on that side tab.
I gave a talk earlier today,
I gave a talk earlier today.
And this was a one of the themes was the two sides of this coin.
Natural foraging systems and engineering forging systems and algorithms in this sort of bi-directional relation in this perspective swap systems as they are and the empirical data, the inspiration they provide.
And then also these computer scientific, more engineered or more mathematical
models of multi-agent settings and the ways that those are kind of coming together,
giving all these different optionalities for modeling.
Okay.
See stunning sites only accessible by rail.
Okay, drone at least has started up.
This one always looks like it's halting but I think at these stages, maybe when it runs the paper solver, even just one time, this could be a lot faster probably with the parallelized labs and with not just this older laptop that I'm running this from.
This seems like there's a lot of back-end
possibilities and from the requirements installation. Like there are some packages.
It's sort of like, oh, I wonder why they're doing that.
And so it seems like there's some pretty
advanced methods that are being used sequentially.
And that was what I sort of broke it trying to get at.
And that, and, and, but what I know will, will be part of it in, in the bigger picture is that kind of expressivity graphical user interface from no to low to very medium and high code, high touch ways of working with these informations.
And these are all just sort of, again, haphazard archetypes.
And, um, the, they will already in their own description of frictions and just looking at functions here as randomly assembled on a given afternoon.
Um, and pointed to by these repos and peers and sort of adjacent from, from this sort of concept.
There's just a lot of space to think about how, how to get this to all work.
And whether this is the best agent orchestration framework or as several times,
I've just built it from scratch with just a few prompts or whether using some other agent orchestration frameworks in any given moment, that's, that's a super important question.
And I, I hope that we continue to develop down those lines and stay multiple there.

Um, there's just sort of overlapping and complimentary functions that different modeling approaches can yield.
And, uh, having the, the modeling capabilities connected with like requirements, engineering,
pattern languages, those kinds of features so that there can be formal and natural language,
um, effective descriptions of systems.
So, connecting that to the compositionality of multi-agent LLM browser type,
code writing and, uh, reviewing, improving the code, which at least the first time that I got it to work,
it, it totally did do sequential code improvement and running.
I don't know with what resourcing it was running the code or how it was getting to the outputs of the code,
um, in terms of its workspace and all of that.
But, but, but those are just the questions.
Um, 20 steps or what, what's the right number of what compositions, okay, content empty.
So, even though while the outputs are in many ways less impressive than just one perplexity,
um, search might get you, it's about these functionalities
and different compositionalities being more and more accessible, more multimodal, um, part of larger orchestration frameworks, more open to calling and receiving API calls and different information measures on the input and the output and the cognitive nature of different systems, um, that sort of low road, high road, um, whether it's as systems, uh, must be for a given observer, all these different ways that people talk about the different,
um, relational modeling stance and pragmatism, how that's all connecting with just the ability to vibe code and, uh, make documentation tests, logging functions, a lot of different things that, okay, this, this finished.
Did it even save a file? Maybe.
But, it did do some browser opening, so, okay. Goodbye, open manis.
So, now we have just in this last minute, see what happens, we have the, uh, agent lab,
which does the, and, and, and prior did get some artifacts being built
of just multi-agent coordination and the RxInfer giving some issues for the drone example, but was much easier to move over to the script form for the neural network and the recurrent, uh, switching linear dynamical systems model. So, thank you all. Hope that was interesting and or useful.
Not as much baseball and so on as there could have been, but c'est la vie. So, thank you till next time. Bye.
Bye.

Thank you.